

TP6 signal



Derridj Mellyna

Loic Weber

Préparation :

On considère le fichier `placement.npz` dont on extrait les infos de la façon suivante pour chaque filtre :

p.1

```
data = load("C:/Users/33658/Documents/placement.npz")

num1a = data['num1a']
den1a = data['den1b']
p1a = roots(den1a)
z1a = roots(num1a)
```

Fig 1.1 Extraction des données de placement.npz

p.2

A l'aide du module numpy `freqz`, on obtient les pulsations et les modules pour chaque filtre :

```
w_1a, H_1a = freqz(num1a, den1a)
w_1b, H_1b = freqz(num1b, den1b)
w_1c, H_1c = freqz(num1c, den1c)
w_2a, H_2a = freqz(num2a, den2a)
w_2b, H_2b = freqz(num2b, den2b)
w_2c, H_2c = freqz(num2c, den2c)
```

Fig 1.2 Obtention des amplitudes et pulsations de chaque filtre

p.3 on affiche alors les réponses impulsionnelles de chaque filtre

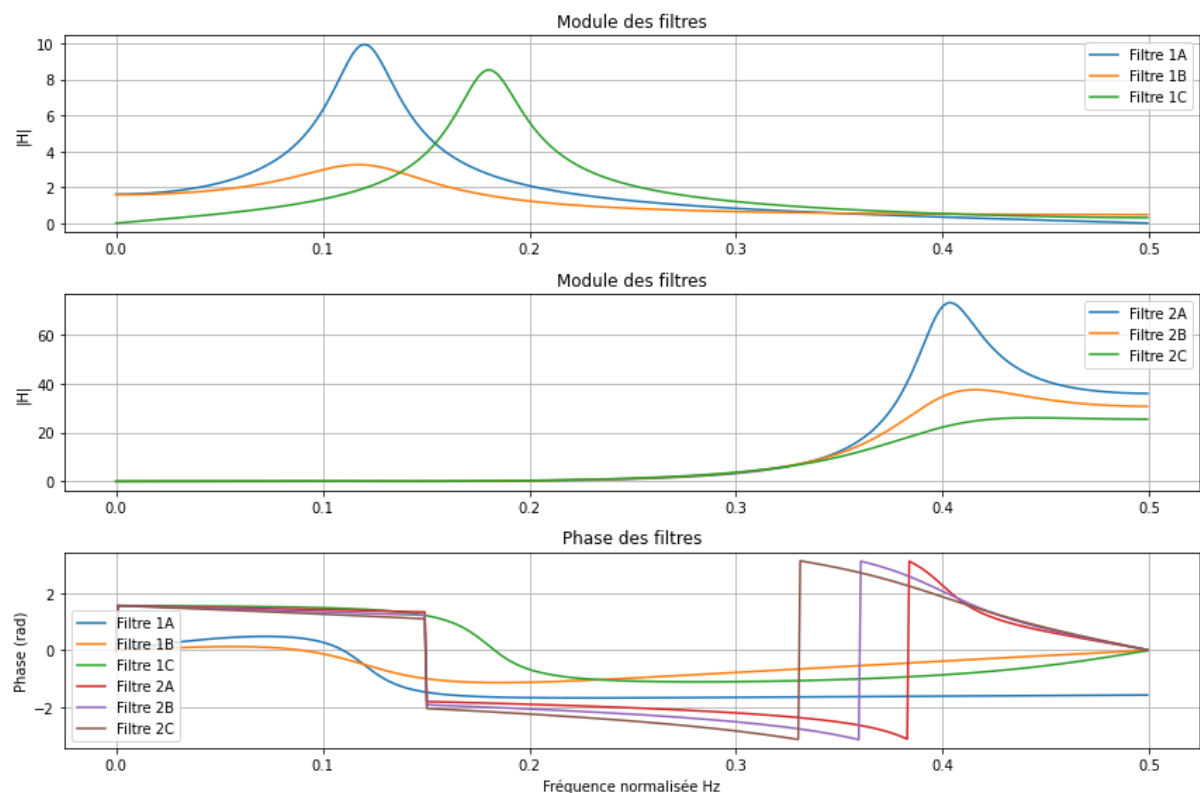


Fig 1.3 Réponses impulsionnelles de chaque filtre.

p.4 Pour faciliter la comparaison de chaque filtre, on normalise à chaque fois par le gain max (pour courbes en gain) :

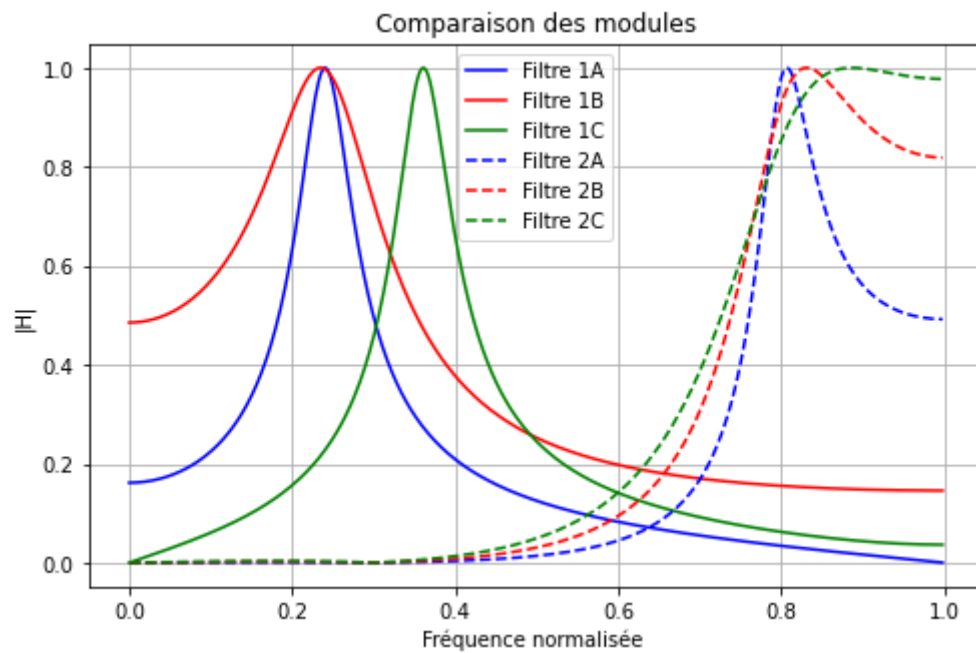


Fig 1.3 Réponses impulsionnelles normalisés de chaque filtre.

p.5

On trace finalement les DPZ des filtres (le code de la fonction de la DPZ est présent en annexe)

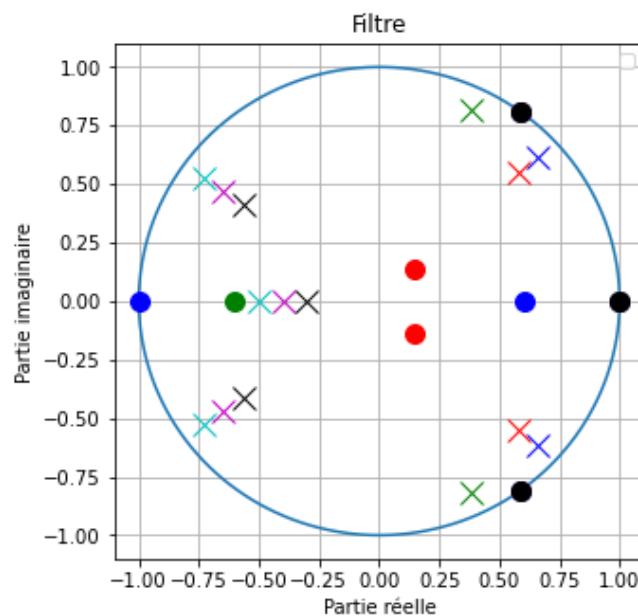


Fig 1.4 DPZ des filtres

p.6 l'ensemble des caractéristiques des filtres est récapitulé dans le tableau ci-dessous :

	Hz1a	Hz1b	Hz1c
zp	-1, 0.6	0.1458+0.1369j, 0.1458-0.1369j	1.0000+0.0000j, -0.6000+0.0000j
 zp 	1.0000, 0.6000	0.2000, 0.2000	1.0000, 0.6000
arg(zp)	3.1416, 0.0000	0.7540, -0.7540	0.0000, 3.1416
pq	0.6561+0.6161j, 0.6561-0.6161j	0.5832+0.5476j, 0.5832-0.5476j	0.3832+0.8143j, 0.3832-0.8143j
 pq 	0.9000, 0.9000	0.8000, 0.8000	0.9000, 0.9000
arg(pq)	0.7540, -0.7540	0.7540, -0.7540	1.1310, -1.1310
Q	5.00	2.50	5.00

	Hz2a	Hz2b	Hz2c
zp	1.0000+0.0000j, 0.5878+0.8090j, 0.5878-0.8090j	1.0000+0.0000j, 0.5878+0.8090j, 0.5878-0.8090j	1.0000+0.0000j, 0.5878+0.8090j, 0.5878-0.8090j
 zp 	1.0000, 1.0000, 1.0000	1.0000, 1.0000, 1.0000	1.0000, 1.0000, 1.0000
arg(zp)	0.0000, 0.9425, -0.9425	0.0000, 0.9425, -0.9425	0.0000, 0.9425, -0.9425
pq	-0.7281+0.5290j, -0.7281-0.5290j, -0.5000+0.0000j	-0.6472+0.4702j, -0.6472-0.4702j, -0.4000+0.0000j	-0.5663+0.4114j, -0.5663-0.4114j, -0.3000+0.0000j
 pq 	0.9000, 0.9000, 0.5000	0.8000, 0.8000, 0.4000	0.7000, 0.7000, 0.3000
arg(pq)	2.5133, -2.5133, 3.1416	2.5133, -2.5133, 3.1416	2.5133, -2.5133, 3.1416
Q	5.00	2.50	1.67

Fig1.5 Récapitulatif des caractéristiques des filtres

p.7

Les filtres 1A et 1B sont des passe-bas : gain élevé en basse-fréquence

Le filtre 1C est un passe-bande à une fréquence de coupure

Les filtres 2A 2B et 2C sont des passe-haut avec différentes valeurs du facteur de qualité. On remarque que le facteur de qualité augmente (en normalisation), plus le phénomène de résonance s'accroît (pic étroit et plus prononcé)

p.8

Quand M passe près d'un zéro, $|H(\exp(j2\pi\nu))|$ chute.

Quand M passe près d'un pôle, $|H(\exp(j2\pi\nu))|$ augmente.

VI.2 Séparation des sources

Objectifs :

- Mettre en œuvre une chaîne de traitement de données
- Caractériser un signal grâce à ses représentations fréquentielle et temporelle
- Lire et écrire des données dans un fichier annexe

2.3 Synthèse des filtres audio et informatif

On considère le fichier source music25.wav . Cet extrait contient de bandes utiles : une bande utile : $\text{audioSource} = [0, 17 \text{ kHz}]$ (contenant un signal audio) et une bande utile $\text{sigInfo} = [17\text{kHz}, 17,5\text{kHz}]$ (contenant les informations du signal).

L'objectif est ici de filtrer cet extrait audio afin d'obtenir l'audio et l'information séparément.

On trace donc la transformée de fourier du signal audio ainsi que sa représentation en temporelle.

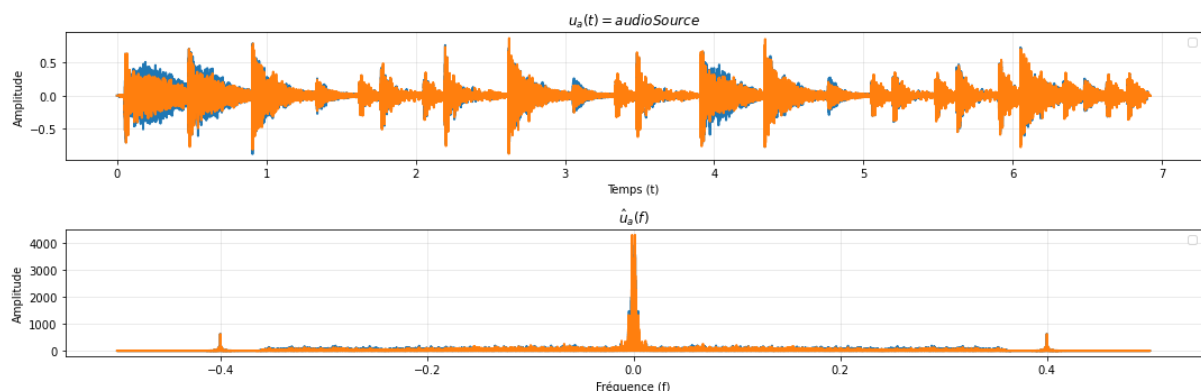


Fig 2.1 audio source et sa fft

On cherche à appliquer un filtre passe-bande (pour extraire audioSource) et un coupe-bande (pour extraire sigInfo).

Objectif (passe-bande) :

- Réaliser un filtre passe-bande dont les zéros sont en 0 et 0.5 et deux pôles en 0.4 et -0.4

Objectif (coupe-bande) :

- Placer 3 zéros et 3 pôles (on en utilise 3 pour éviter une ondulation dans la zone d'entre deux)

```

zero1=1
zero2=-1

pole1=0.99*exp(2j*pi*0.4)
pole2=0.99*exp(-2j*pi*0.4)

num=poly([zero1,zero2])
den=poly([pole1,pole2])

w, H = freqz(num, den)

theta = linspace(0, 2*pi, 500)
figure(figsize=(5,5))
plot(cos(theta), sin(theta))
plot(real(zero1), imag(zero1), 'ob', markersize=10)
plot(real(zero2), imag(zero2), 'ob', markersize=10)
plot(real(pole1), imag(pole1), 'xr', markersize=12)
plot(real(pole2), imag(pole2), 'xr', markersize=12)
xlabel('Partie réelle')
ylabel('Partie imaginaire')
title('Filtre')
grid()
legend()
show()

```

Fig 2.2 Pôles et zéros pour le passe-bande (code)

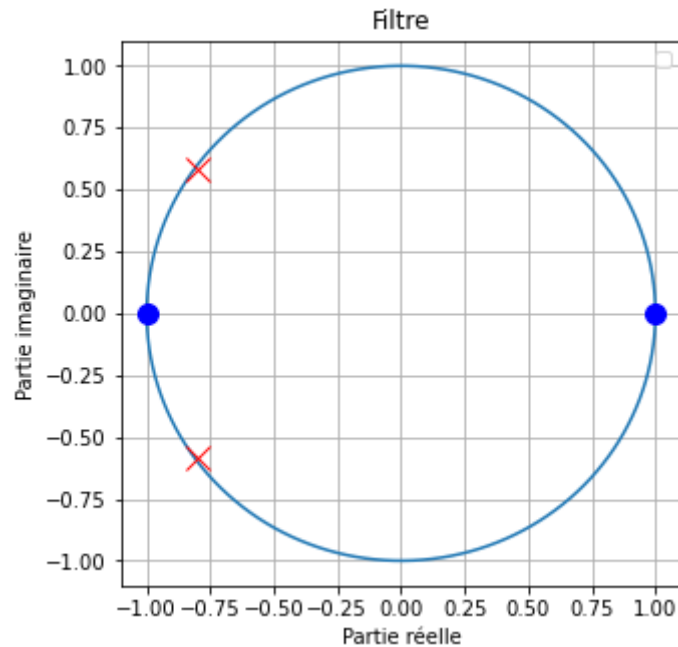


Fig 2.3 Positionnement des poles et des zéros

Après avoir normalisé par le gain max, on obtient le filtre passe-bande suivant :

```

figure(figsize=(8,5))
plot(w/(2*pi), abs(H)/max(abs(H)), label='Filtre passe-bande')
xlabel('Fréquence normalisée')
ylabel('|H|')
title('filtre passe-bande')
grid()
legend()
show()

```

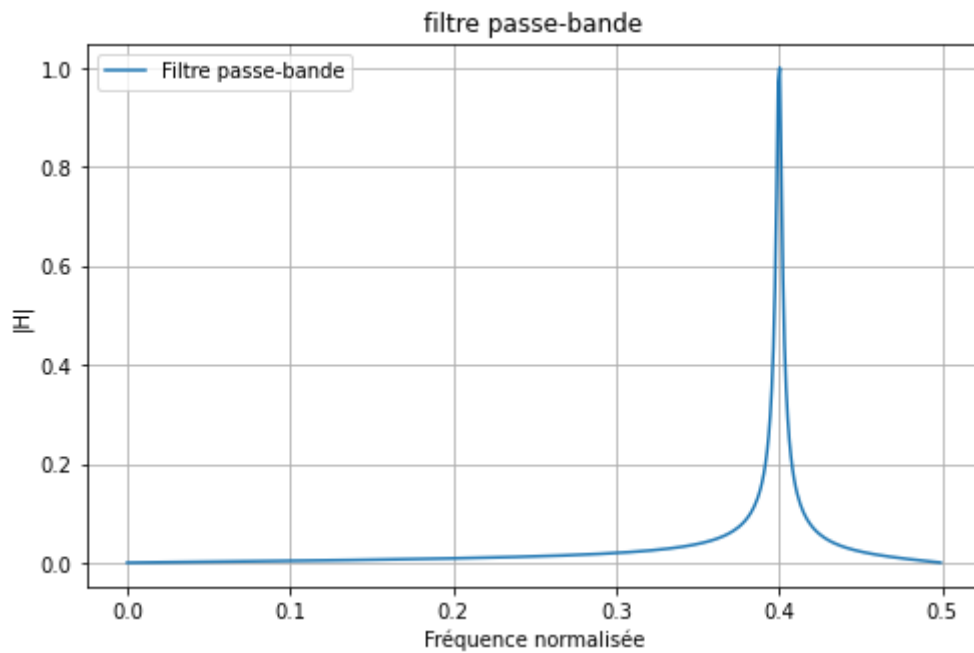


Fig 2.4 Filtre passe-bande

On réalise le code suivant qui permet de générer la sortie (une fois l'entrée filtrée). On affiche ensuite le signal une fois filtrée ainsi que la fft de la sortie. (On se contente ici de donner uniquement le code pour le passe-bande mais le principe du coupe-bande reste le même)

```
Gmax = max(abs(H))

sortie=zeros_like(audio_source)

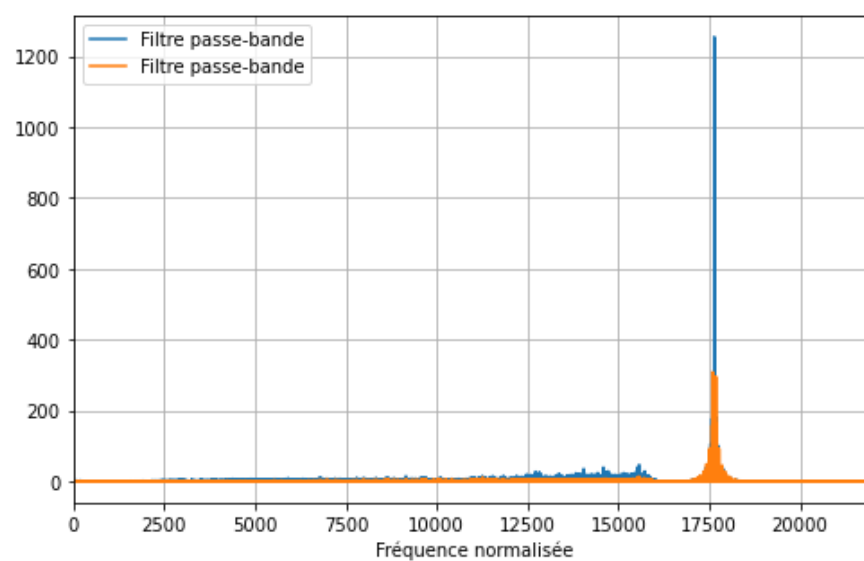
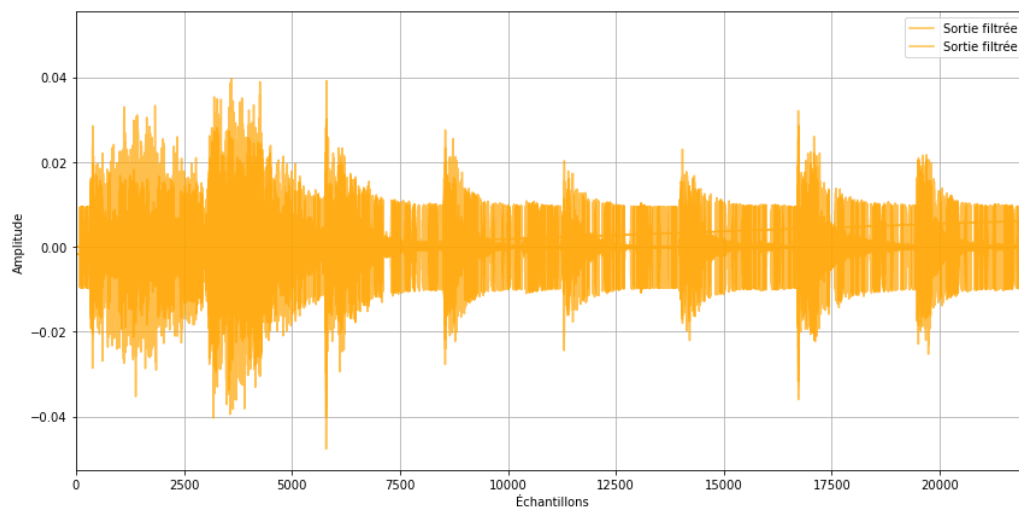
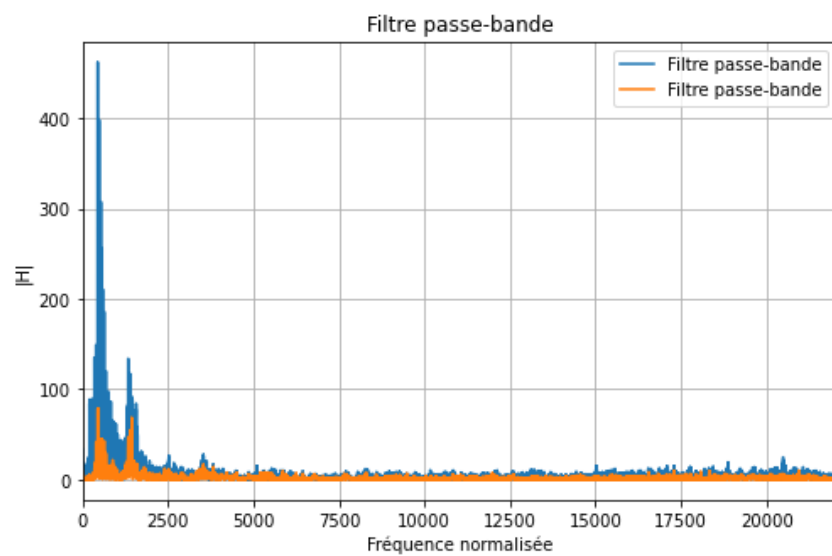
sortie[:,0] = lfilter(num/max(abs(H)), den, audio_source[:,0])
sortie[:,1] = lfilter(num/max(abs(H)), den, audio_source[:,1])
# --- 3. Affichage des signaux ---
figure(figsize=(12, 6))

plot(f, sortie, label="Sortie filtrée", color="orange", alpha=0.7)
xlabel("Échantillons")
ylabel("Amplitude")
xlim(0, fe/2)
legend()
grid(True)

tight_layout()
show()

figure(figsize=(8,5))
plot(f, abs(fft2(sortie)), label='Filtre passe-bande')
xlabel('Fréquence normalisée')
xlim(0, fe/2)
grid()
legend()
show()
```

On obtient les résultats suivant pour le passe-bande :



On obtient les résultats suivant pour le coupe-bande :

